# Mastering Git — Hands-On Lab Guide

Companion labs for the "Mastering Git" presentation

6 Labs | Beginner to Advanced | Team Practice

2026 | Generated by Genspark AI

## Introduction

These labs are designed to give your team hands-on experience with Git concepts covered in the Mastering Git presentation. Complete them in order — each lab builds on the previous one.

### Prerequisites

| Requirement | Details |
|---|---|
| Git installed | `git --version` should return 2.x+ |
| Code editor | VS Code recommended |

| Requirement | Details |
|---|---|
| GitHub/Azure DevOps account | Free account required for Lab 4–6 |
| Terminal access | Git Bash (Windows) or Terminal (Mac/Linux) |

## Lab Overview

| Lab | Title | Difficulty | Est. Time |
|---|---|---|---|
| Lab 1 | Git Setup & Your First Commit | 🟢 BEGINNER | 15 mins |
| Lab 2 | Branching & Merging | 🟢 BEGINNER | 20 mins |
| Lab 3 | Stash, Reset & Revert | 🟡 INTERMEDIATE | 20 mins |
| Lab 4 | The Pull Request Workflow | 🟡 INTERMEDIATE | 25 mins |
| Lab 5 | Git Flow — Enterprise Branching | 🔴 ADVANCED | 30 mins |
| Lab 6 | Linking Commits to Work Items | 🔴 ADVANCED | 20 mins |

# 01 Git Setup & Your First Commit

🟢 BEGINNER ⏱ 15 mins

**Objective:** Configure Git, initialize a repo, and make your first commit.

## Step 1 — Verify Installation

Copy

```
git --version
```

✅ Expected output: "git version 2.x.x"

## Step 2 — Configure Your Identity

Copy

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --list
```

✅ Expected: Your name and email appear in the output.

## Step 3 — Create a New Repository

Copy

```
mkdir my-git-lab
cd my-git-lab
git init
```

✅ Expected: Initialized empty Git repository in .../my-git-lab/.git/

## Step 4 — Create Your First File

Copy

```
echo "# My Git Lab" > README.md
echo "Learning Git the right way!" >> README.md
git status
```

✅ Expected: README.md shows as Untracked file in red.

## Step 5 — Stage and Commit

Copy

```
git add README.md
git status
git commit -m "Initial commit: Add README"
git log
```

✅ Expected: Commit appears in log with your name, email, and message.

## Step 6 — Create a .gitignore

Copy

```
echo "*.log" > .gitignore
echo "bin/" >> .gitignore
echo "obj/" >> .gitignore
git add .gitignore
git commit -m "Add .gitignore for build artifacts"
```

**Lab 1 Checkpoint:**

- [ ] Git is installed and configured
- [ ] Repository initialized
- [ ] Two commits visible in git log

- [ ] .gitignore created

💡 **Key Takeaway:** Every Git workflow starts with `init` or `clone`, then the cycle: **Edit** → **Stage (add)** → **Commit**. You're always working locally first!

# 02 Branching & Merging

🟢 BEGINNER ⏱ 20 mins

**Note:** Continues from Lab 1 — use the same `my-git-lab` repo.

**Objective:** Create a feature branch, make changes, and merge back to main.

## Step 1 — Check Branch

Copy

```
git branch
```

✅ Expected: * main

## Step 2 — Create Feature Branch

Copy

```
git checkout -b feature/add-homepage
git branch
```

✅ Expected: * feature/add-homepage active

## Step 3 — Create Files

Copy

```
echo "<h1>Welcome</h1>" > index.html
git add index.html
git commit -m "Add index.html"

echo "body { background: #fff; }" > style.css
```

```
git add style.css
git commit -m "Add style.css"
```

### Step 4 — Verify Log

Copy

```
git log --oneline
```

✅ Expected: 2 new commits on feature branch

### Step 5 — Switch Back

Copy

```
git checkout main
ls
```

❗ Notice: index.html is NOT here (only on feature branch) — important insight!

### Step 6 — Merge

Copy

```
git merge feature/add-homepage
ls
git log --oneline
```

✅ Expected: files now on main

### Step 7 — Cleanup

Copy

```
git branch -d feature/add-homepage
```

```
git branch
```

**Lab 2 Checkpoint:**

- [ ] Feature branch created with correct naming convention
- [ ] Changes confirmed NOT visible on main before merge
- [ ] Merge completed successfully
- [ ] Branch deleted after merge

💡 **Key Takeaway:** Branches are cheap — use them for everything! A branch is just a lightweight pointer. Create one for every feature, bug fix, or experiment.

# 03 Stash, Reset & Revert

🟡 INTERMEDIATE ⏱ 20 mins

**Objective:** Practice the three most important 'undo' tools in Git.

## Part A — git stash (Save Work Without Committing)

**Scenario:** You're mid-way through a feature when your manager asks you to urgently fix a bug. You can't commit half-done work. Use stash!

Copy

```
# 1. Start new work
echo "WIP code" > wip.txt
git add wip.txt

# 2. Stash it
git stash
git status

# 3. Simulate urgent fix
echo "Urgent fix" > fix.txt
git add fix.txt
git commit -m "Urgent fix applied"

# 4. Bring back work
git stash pop
```

## Part B — git revert (Safe Undo — Keeps History)

**Scenario:** You committed something wrong. Use revert to safely undo it without destroying history — safe for shared branches!

Copy

```
# 1. Make bad commit
echo "BAD CODE" > mistake.txt
```

```
git add mistake.txt
git commit -m "Accidental bad commit"

# 2. Revert it
git revert HEAD
```

✅ Expected: A NEW commit appears saying "Revert accidental bad commit"

### Part C — git reset --hard (⚠️ Destructive — Local Only!)

⚠️ **WARNING:** This CANNOT be undone! Only use on uncommitted local changes.

Copy

```
echo "Garbage code" > junk.txt
git add junk.txt
git status

# DESTROY changes
git reset --hard HEAD
git status
```

**Lab 3 Checkpoint:**

- [ ] Successfully stashed and restored work
- [ ] git revert created a new reverting commit
- [ ] git reset --hard discarded uncommitted changes
- [ ] Understand the difference: revert = safe, reset = destructive

💡 **Key Takeaway:** Rule of thumb: Use `revert` on shared/public branches. Use `reset --hard` only on local uncommitted changes. Use `stash` when switching context without committing.

# 04 The Pull Request Workflow

🟡 INTERMEDIATE ⏱ 25 mins Requires GitHub/Azure DevOps

**Objective:** Push a branch to GitHub and open a Pull Request for review.

### Step 1 — Create Remote Repo

Go to GitHub.com, create new repo "git-lab-practice". Do NOT initialize with README.

### Step 2 — Connect & Push

Copy

```
git remote add origin https://github.com/YOUR_USER/git-lab-practice.git
git branch -M main
git push -u origin main
```

### Step 3 — Feature Branch

Copy

```
git checkout -b feature/contact-page
echo "Contact Us" > contact.md
git add contact.md
git commit -m "Add contact page"
git push -u origin feature/contact-page
```

### Step 4 — Open Pull Request

1. Go to GitHub repo page.
2. Click "Compare & pull request".
3. Add title & description.
4. Click "Create pull request".

### Step 5 — Merge & Sync

1. Click "Merge pull request" on GitHub.

2. Back in terminal:

```
Copy
git checkout main
git pull
git branch -d feature/contact-page
```

**Lab 4 Checkpoint:**

- [ ] Remote repository connected
- [ ] Feature branch pushed to GitHub
- [ ] Pull Request opened with description
- [ ] PR merged and branch deleted
- [ ] Local main synced with git pull

💡 **Key Takeaway:** Pull Requests are not a Git feature — they're a platform feature. They add a collaboration, review, and approval layer on top of Git's merging capability.

# 05 Git Flow — Enterprise Branching

🔴 ADVANCED ⏱ 30 mins

**Objective:** Implement the full Git Flow model with main, develop, and feature branches.

```
main: ●──────────────────────────────────────────● (v1.0.0) \ /
develop: ●──────●──────────────●──────────────────/ \ / \ / feature:
●──────────● ●──────────────●
```

## Step 1 — Setup Develop

Copy

```
git checkout main
git checkout -b develop
git push -u origin develop
```

## Step 2 — Feature Workflow

Copy

```
git checkout -b feature/user-dashboard develop
# ... do work ...
git commit -m "feat: Add dashboard"
git push origin feature/user-dashboard
```

*Create PR targeting **develop** branch. Merge it.*

## Step 3 — Release

Copy

```
git checkout develop
git pull
git checkout main
git merge develop
```

```
git tag -a v1.0.0 -m "Release v1.0"
git push origin main --tags
```

### Step 4 — Visualize

Copy

```
git log --oneline --graph --all
```

**Lab 5 Checkpoint:**

- [ ] develop branch created and pushed
- [ ] Feature branch created from develop (not main!)
- [ ] PR merged into develop
- [ ] develop merged to main for release
- [ ] Version tag created

💡 **Key Takeaway:** Git Flow in practice: Main = production code. Develop = next release staging. Feature = individual tasks. All features merge into develop via PRs. Develop merges to main at release time.

# 06 Linking Commits to Work Items

🔴 ADVANCED ⏱ 20 mins

**Objective:** Link Git commits to project management tasks using commit message conventions.

**Background:** Including `#ID` in commit messages allows platforms like GitHub, Azure DevOps, and Jira to automatically link code to tasks.

### Step 1 — Create Issue

Go to GitHub Issues tab > New Issue > Title "Add Footer". Note the Issue Number (e.g., #5).

### Step 2 — Code & Link

Copy

```
git checkout -b feature/add-footer
echo "Footer" >> index.html
git add index.html
git commit -m "feat: Add footer - closes #5"
git push origin feature/add-footer
```

✨ **Magic Keywords:** `closes #ID`, `fixes #ID`, `resolves #ID` → automatically closes the issue when merged!

### Step 3 — Verify

Open PR. You should see the issue linked in the description. Merge the PR. Check the Issue tab — it should be Closed!

**Lab 6 Checkpoint:**

- [ ] GitHub Issue created
- [ ] Branch created referencing the issue
- [ ] Commit message includes closes #ID syntax
- [ ] Issue automatically closed after PR merge
- [ ] Full traceability: Issue ↔ Commit ↔ PR linked

💡 **Key Takeaway:** Automated Traceability = no more 'what was this change for?' By linking commits to issues/work items, you create an audit trail from business requirement → code change → release.

# Quick Reference Card

## Most Used Commands

| | |
|---|---|
| `git init` | Initialize new repo |
| `git clone` | Download remote repo |
| `git status` | See changed files |
| `git add .` | Stage all changes |
| `git commit -m` | Save local snapshot |
| `git push` | Upload to remote |
| `git pull` | Download + merge remote |
| `git checkout -b` | Create & switch branch |
| `git merge` | Merge branch into current |
| `git stash` | Temporarily save changes |
| `git stash apply` | Restore stashed changes |
| `git log --oneline` | Compact commit history |
| `git revert HEAD` | Safe undo last commit |
| `git reset --hard HEAD` | ⚠️ Discard local changes |

## Branch Naming Conventions

| | |
|---|---|
| `feature/description` | New features |
| `bugfix/description` | Bug fixes |
| `hotfix/description` | Urgent prod fixes |
| `release/version` | Release preparation |

## Commit Message Format

Copy

```
type: Short description (max 72 chars) #issue-id

Types: feat | fix | docs | style | refactor | test | chore
```

# My Lab Notes

Use this space to record observations, questions, and discoveries during the labs

### Lab 01 & 02

### Lab 03 & 04

### Lab 05 & 06